

Ejercicio 11: Maximización de Suma Condicional (Enfoque Voraz Correcto)

Enunciado

Sean dos secuencias de n números naturales cada una, $A = (a_1, a_2, \dots, a_n)$ y $B = (b_1, b_2, \dots, b_n)$, la primera de las cuales, A , está ordenada en forma creciente ($a_1 \leq a_2 \leq \dots \leq a_n$).

Se trata de diseñar un algoritmo que obtenga una reordenación de la secuencia B (llamémosla B') de forma que maximice la suma de los elementos de A que no superan al elemento correspondiente de B' . Es decir, maximizar:

$$\sum_{i|a_i \leq b'_i} a_i$$

Ejemplo:

- $A = (2, 6, 7, 10, 22)$
- $B = (15, 1, 7, 5, 2)$
- La reordenación óptima es $B' = (5, 2, 7, 15, 1)$
- Beneficio obtenido: $2 + 7 + 10 = 19$ (ya que $2 \leq 5$, $7 \leq 7$ y $10 \leq 15$).

1. Modelado e Idea Voraz Correcta

Para maximizar la suma de los valores a_i que logramos cubrir, debemos dar prioridad a cubrir los elementos **más grandes** de A (ya que son los que aportan más puntos a la suma). Por tanto, la estrategia óptima consiste en realizar un emparejamiento de **mayor a menor**:

1. **Ordenar** tanto A como B en orden decreciente (de mayor a menor).
2. Procesar la secuencia B ordenada de mayor a menor (nuestro recurso más potente primero):
 - Para cada $b \in B$, intentamos cubrir el **mayor** elemento disponible de A tal que $a \leq b$.
 - Si existe un a que cumple la condición, realizamos el emparejamiento. Esto asegura que gastamos la "capacidad" de nuestro b en obtener el mayor beneficio posible disponible.
 - Si no existe ningún $a \leq b$, significa que este b (y todos los restantes, que son aún menores) no pueden cubrir a ninguno de los a disponibles. Por tanto, este b es "inútil" y lo emparejamos con el **mayor** a restante (que ya sabemos que es imposible de cubrir) para descartarlo sin malgastar elementos útiles.

2. Algoritmo Voraz (Búsqueda Descendente)

Utilizando vectores ordenados, podemos implementar este proceso de manera muy eficiente usando punteros.

Pseudocódigo

```
Entrada: A[1..n] (ordenada creciente), B[1..n] (desordenada)

// 1. Crear copias ordenadas de forma decreciente (de mayor a menor)
A_dec = Ordenar DECREciente(A) // Ejemplo: [22, 10, 7, 6, 2]
B_dec = Ordenar DECREciente(B) // Ejemplo: [15, 7, 5, 2, 1]

// 2. Inicializar arrays auxiliares para marcar elementos emparejados
B_prime = Array de tamaño n vacío
asignado_A = Array de booleanos de tamaño n inicializado en Falso

// Puntero para recorrer A_dec
idx_A = 1

Para i desde 1 hasta n:
    b_actual = B_dec[i]

    // Avanzar idx_A hasta encontrar el primer elemento de A que b_actual pueda c
    Mientras (idx_A <= n y (A_dec[idx_A] > b_actual o asignado_A[idx_A] == Verdac
        idx_A = idx_A + 1

    Si (idx_A <= n):
        // Emparejamiento exitoso: b_actual cubre a A_dec[idx_A]
        // Guardamos b_actual en la posición original que le corresponde a ese el
        pos_original_A = buscar_posicion_original(A_dec[idx_A])
        B_prime[pos_original_A] = b_actual
        asignado_A[idx_A] = Verdadero
        // Reiniciamos idx_A al principio para el siguiente elemento de B
        idx_A = 1
    Sino:
        // b_actual no puede cubrir a ningún elemento disponible de A.
        // Lo "desechamos" emparejándolo con el mayor A disponible que no esté as
        pos_mayor_libre = obtener_primer_libre(asignado_A)
        pos_original_A = buscar_posicion_original(A_dec[pos_mayor_libre])
        B_prime[pos_original_A] = b_actual
        asignado_A[pos_mayor_libre] = Verdadero
        idx_A = 1

Retornar B_prime
```

3. Análisis de Coste

- **Temporal:**
 - **Ordenación de las secuencias:** $O(n \log n)$.
 - **Bucle de emparejamiento:** $O(n^2)$ en el peor caso si buscamos de manera lineal, pero optimizable a $O(n \log n)$ si usamos estructuras de datos de búsqueda como un árbol binario balanceado (o un `std::set` en C++) para encontrar el elemento menor o igual de forma logarítmica.

- **Coste Total:** $O(n \log n)$.
- **Espacial:** $O(n)$ para almacenar las estructuras auxiliares y el array resultado.